

人工智慧導論



Java

類別與物件

本章提要

- **Java**的類別與物件 - 封裝
- 類別的建構子
- 使用物件的成員資料
- 在物件使用類別變數與方法

Java的類別與物件 – 封裝

- **Java**的類別是用來建立物件，它是一種使用者自行定義的資料型態。(角色定義)
- **Java**物件使用變數儲存狀態稱為「屬性」(Property)或「實例變數」(Instance Variables，也稱為副本變數)(角色的衣著、性別等)，各種行為的程序和函數，在**Java**稱為方法(Methods)。(角色的能力、動作)

類別與物件-說明

- **Java**類別宣告是物件的原型宣告，對於物件原型宣告的類別可以分為**2**個部分，如下所示：
 - 成員資料（**Data Member**）：物件的資料部分，屬於基本資料型態的變數、常數或其它的物件的「成員變數」（**Member Variables**）。
 - 成員方法（**Method Member**）：物件的操作部分，即程式與函數，也就是**Java**的方法。

類別與物件-類別宣告語法

- Java類別宣告語法，如下所示：

class 類別名稱 {

存取敘述修飾子 資料型態 成員資料;

.....

存取敘述修飾子 傳回值型態 成員方法(參數列) {
 程式敘述;

}

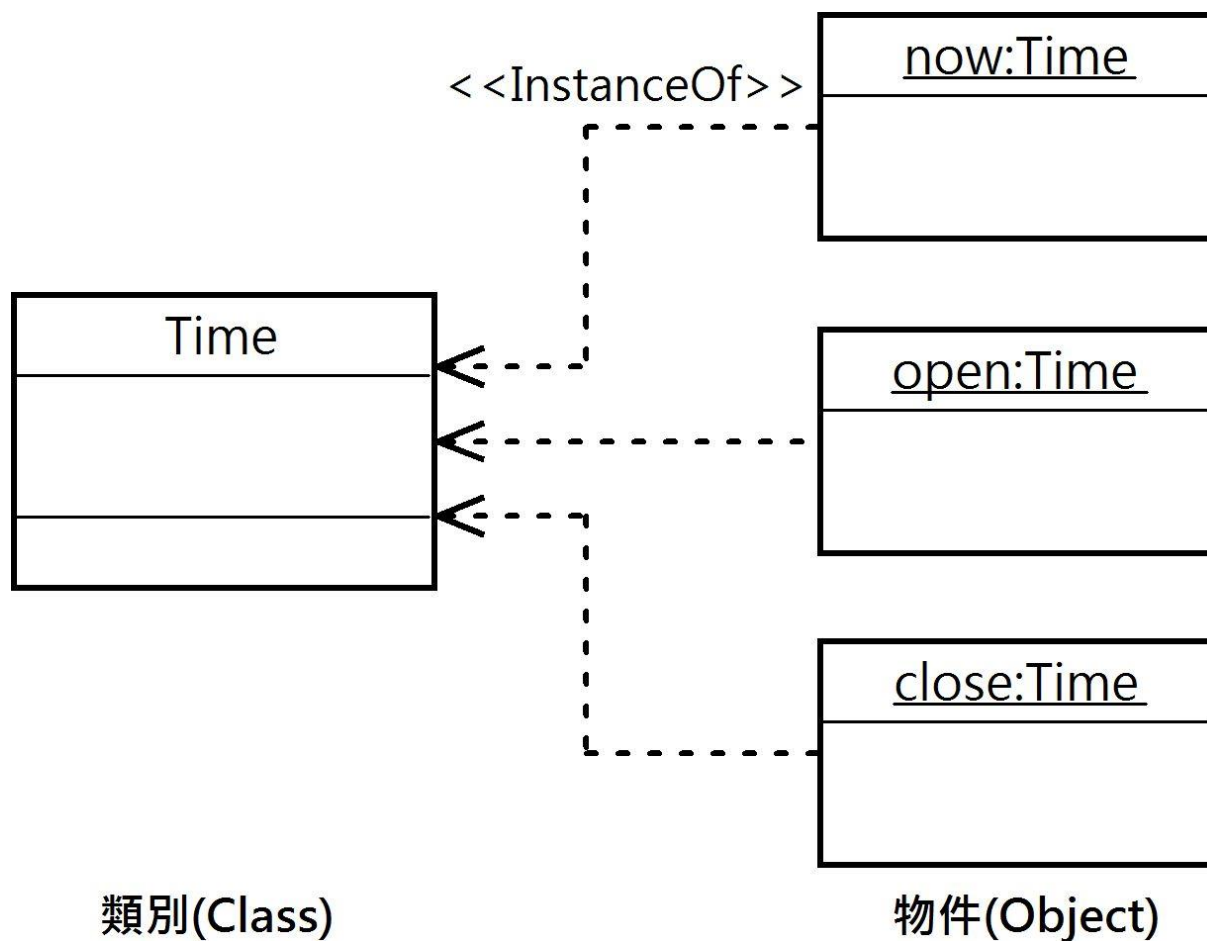
.....

}

類別與物件-類別與物件(說明)

- 在Java程式是使用類別原型建立物件，稱此物件屬於類別的「實例」(**Instance**)或稱為副本，物件的變數稱為「實例變數」(**Instance Variables**)，物件的程式或函數稱為「實例方法」(**Instance Methods**)。
- 同一個類別可以當作範本建立無數個物件實例，每一個物件都屬於類別的實例，或直接稱為物件。

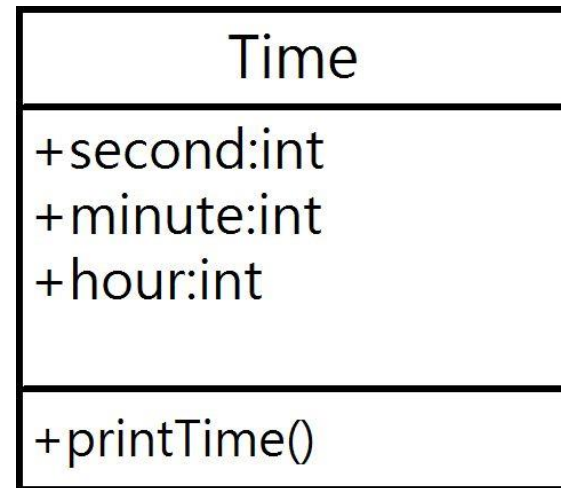
類別與物件-類別與物件(圖例)



類別與物件-範例

- 例如：將各種時間資料的物件抽象化成類別**Time**的物件原型宣告，如下所示：

```
class Time {  
    public int second;  
    public int minute;  
    public int hour;  
    public void printTime() {
```



```
        System.out.println(hour+": "+minute+": "+second);  
    }  
}
```


類別與物件-宣告物件變數(說明)

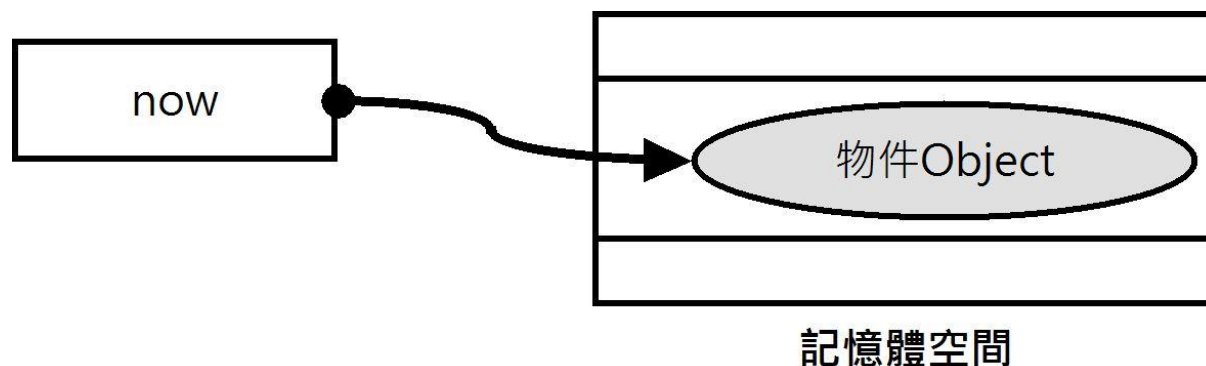
- 在宣告**Java**類別後，我們就可以建立物件，首先將類別當作一種新的資料型態，使用新資料型態宣告所需的物件變數，如下所示：

Time now, open, close;

- 程式碼宣告**Time**類別的變數**now**、**open**和**close**，稱為物件變數，變數內容是參考的指標，這個指標就是物件的識別字（**Identity**），目前我們只有宣告物件變數，並沒有真正建立物件。

類別與物件-宣告物件變數(圖例)

- **Java**物件變數的內容並不是物件本身，物件變數是儲存物件「**參考**」（**Reference**）的**指標**，所謂建立物件是配置一塊記憶體空間儲存物件的內容（實例變數和方法），類別宣告的物件變數儲存的是這塊記憶體空間的位址，告訴我們如何找到此物件，如下圖所示：



類別與物件-建立物件實例

- 在Java建立物件需要使用new運算子，這個運算子可以依照類別的範本來建立物件，傳回指向此物件的參考指標，此過程稱為「實體化」（Instantiation），將類別實體化成物件實例，如下所示：

```
close = new Time();
```

- 使用new運算子建立Time類別的物件實例，物件變數close的值並不是物件本身，而是參考到此物件的指標。

類別與物件-存取實例變數與方法 (說明)

- 在建立好物件後，就可以存取物件的實例變數與方法，其語法如下所示：

物件.實例變數

物件.實例方法

- 物件的實例變數和方法存取都是使用”.”運算子。
 -

類別與物件-存取實例變數與方法 (範例)

- 以**Time**類別建立的物件為例，如下所示：
`close.hour = 22;`
`close.minute = 30;`
`close.second = 30;`
- 指定敘述指定物件變數**close**參考物件的實例變數**hour**、**minute**和**second**的值。同樣的語法可以呼叫實例方法，如下所示：
`close.printTime();`

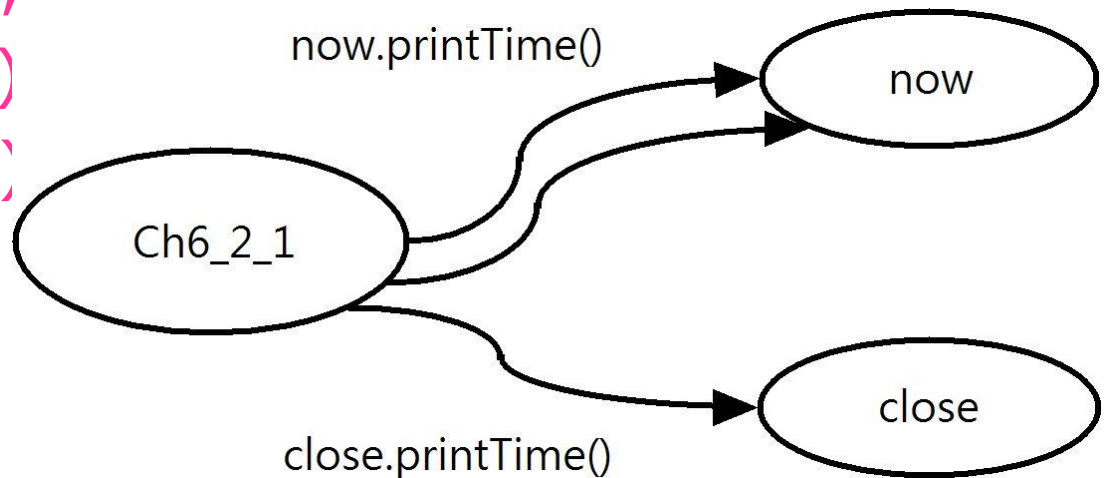
類別與物件-存取實例變數與方法 (圖例)

- 類別可以建立多個物件實例，每一個物件都可以呼叫自己的實例方法或存取實例變數，而所謂執行方法就是送出不同的訊息，如下所示：

`now.printTime();`

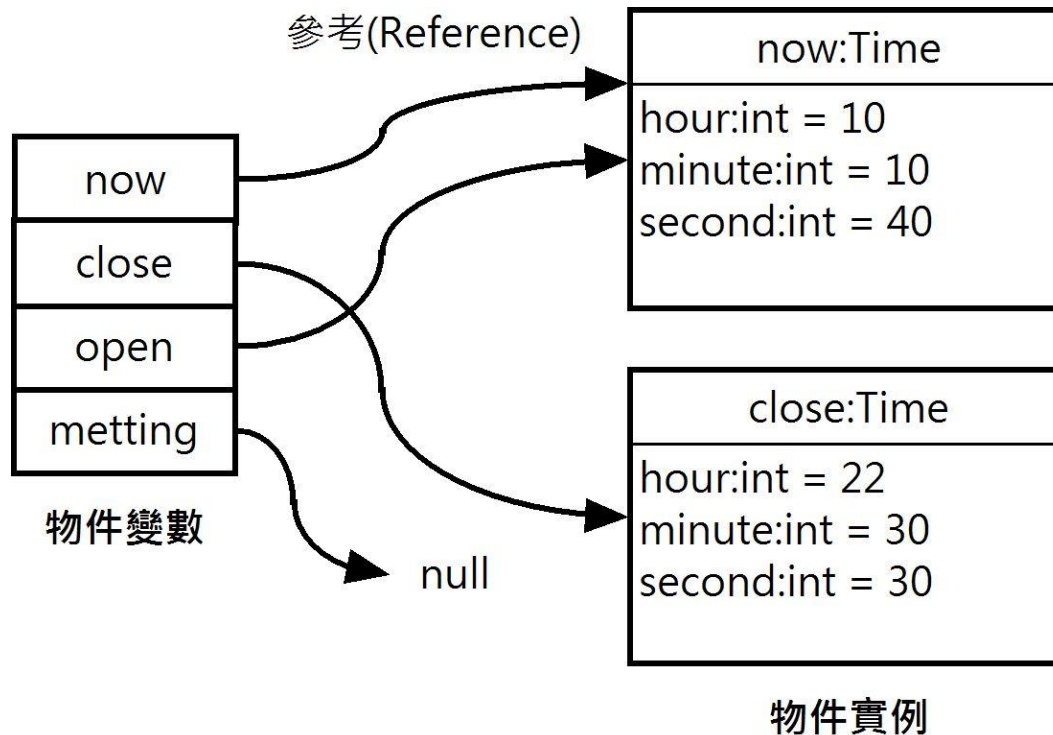
`open.printTime()`

`close.printTime()`



類別與物件

- 在執行Time的Java程式後，電腦記憶體空間的內容，如下圖所示：



成員資料的存取 - 資訊隱藏(說明)

- **Java**類別宣告的成員資料或方法可以使用 **private**和**public**兩種修飾子來指定其存取方式，其說明如下所示：
 - **private**修飾子：成員資料或成員方法只能在類別本身呼叫或存取。
 - **public**修飾子：成員資料或方法是此類別建立物件對外的使用介面，可以讓其他類別的程式碼呼叫此物件的實例方法或存取實例變數。

成員資料的存取 - 資訊隱藏(範例)

- 例如：將**Time**類別的成員資料都宣告成**private**，然後透過宣告成**public**的成員方法**setTime()**和**printTime()**來進行存取，如下所示：

```
class Time {  
    private int second;  
    private int minute;  
    private int hour;  
    public boolean setTime(int h, int m, int s) { }  
    public void printTime() { }  
}
```

物件的使用介面 - 成員方法(說明)

- 在**Java**類別的成員方法如果宣告成**public**，表示這些方法是物件的使用介面，但是有一些方法，例如：**Time**範例的**validateTime()**方法因為只有類別本身在呼叫，我們只需宣告成**private**讓類別本身使用即可，這種方法稱為「工具方法」（**Utility Methods**）。

物件的使用介面 - 成員方法(範例)

- **Time**範例只提供設定時間資料的**setTime()**方法，在這一節的程式範例將新增取得時間資料的方法，如下所示：

```
public int getHour(){ return hour; }  
public int getMinute(){ return minute; }  
public int getSecond(){ return second; }
```

- 3個方法可以取得物件時間資料的小時、分鐘和秒數，此時的類別成員資料已經擁有完整的存取介面，至於成員資料的資料型態是什麼？已經不重要！因為資料本身已經被類別完全封裝起來。

類別的建構子-說明

- **Java**基本資料型態在宣告變數時會配置所需的記憶體空間，至於類別型態的變數並不會自動建立物件，如果類別沒有建構子，在使用**new**運算子建立物件時，只會配置所需的記憶體空間，但是，並不會指定實例變數的初值。
- 如果希望如同基本資料型態在宣告時能夠指定變數初值，在類別宣告需要新增建構子，建構子是物件的初始方法，在建立物件時就會自動呼叫此方法。

類別的建構子-物件壽命

- 從物件使用**new**運算子建立物件到物件不再使用的期間，稱為「物件壽命」（**Object Lifetime**），因為**Java**擁有垃圾收集（**Garbage Collection**）功能負責處理不再使用的物件，程式設計者並不用自行處理，只需考量如何建立類別的建構子。

類別的建構子-特點

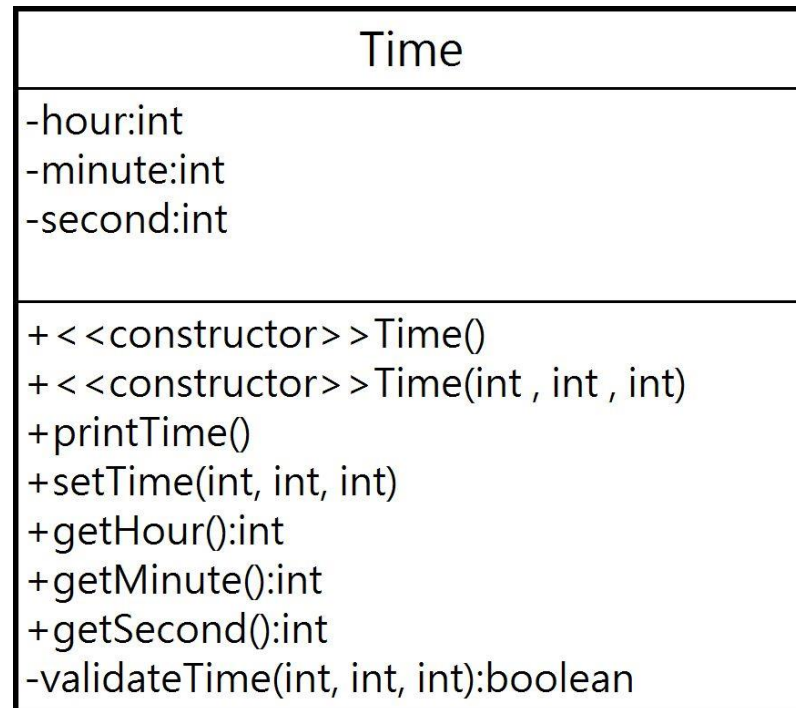
- 建構子與類別同名，例如：類別**Time**的建構子方法是**Time()**。
- 建構子沒有傳回值。
- 建構子支援方法的「過載」（**Overload**），也就是說可以擁有多個同名的建構子方法，只是擁有不同的參數型態和參數個數，如下所示：

Time();

Time(int h, int m, int s);

類別的建構子-UML類別圖

- 在UML類別圖的建構子也是位在方法清單，只是在前面加上<<constructor>>，如下圖所示：



使用this參考物件本身-使用在方法

- 在類別的成員方法可以使用**this**關鍵字參考物件本身的實例方法和變數，例如：參數列如果和實例變數名稱相同，我們可以使用**this**關鍵字指明是實例變數，如下所示：

```
public boolean setTime(int hour,int minute,int
    second) {
    if ( validateTime(hour, minute, second) ) {
        this.hour = hour; this.minute = minute;
        this.second = second;
        return true;
    } else return false;
}
```


使用this參考物件本身-使用在建構子

- 另一種**this**常用的地方是建構子方法，如下所示：

```
public Time() { this(12, 30, 10); }  
public Time(int m) { this(12, m, 10); }  
public Time(int h, int m) { this(h, m,  
    10); }  
public Time(int hour, int minute, int  
    second) {  
    .....  
}
```

使用this參考物件本身-串流呼叫方法

- 因為setTime()方法的傳回值是Time物件，即return this;（Java SE 7版宣告成void的方法也視為傳回物件本身），所以，我們可以使用「串流呼叫方法」（Method Chaining），如同項鍊的一串珠子一般依序呼叫各方法，如下所示：

```
close.setTime(23, 10, 20).printTime();
```

- 程式碼首先呼叫setTime()方法指定時間，因為傳回值是Time物件close，所以可以再呼叫此物件的printTime()方法顯示時間資料。

使用物件的成員資料-說明

- **Java**類別的成員資料除了使用基本資料型態的變數外，也可以使用其它物件的物件變數，例如：**Customer**類別宣告中有**Date**物件，如下所示：

```
class Customer {  
    private int id;  
    private char rank;  
    private Date birthday;  
    public Customer(int id,char r,int m,int d,int y)  
    {  
    }  
    public void printCustomer() {  
    }  
}
```

使用物件的成員資料-注意事項

- 請注意！物件變數的內容只是指向物件的參考指標，並沒有真正建立物件，所以建構子需要使用 **new** 運算子建立物件，如下所示：

```
birthday = new Date(m, d, y);
```

在物件使用類別變數與方法-說明

- 在本節之前的類別宣告都沒有使用**static**宣告的變數和方法，因為在類別宣告**static**的類別變數和方法後，建立物件並不會替每一個物件實例建立獨立的類別變數和方法，只有不是宣告成**static**部分才會建立獨立的實例變數和方法。
- 類別變數和方法是屬於類別，所有物件都是使用同一份類別變數和呼叫同一個類別方法。

在物件使用類別變數與方法-範例

- 例如：**Student**類別擁有類別變數**teacherNo**、**count**和類別方法**getStudentCount()**，如下所示：

```
class Student {  
    public static int teacherNo = 500;  
    private static int count;  
  
    .....  
    public static int getStudentCount() { return  
count; }  
    .....  
}
```

在物件使用類別變數與方法- UML類別圖

- 在執行上一頁Java程式建立的UML類別和物件圖，如下圖所示：

